Track: AI & Automation (Beyond Wrappers)

1) Problem Statement Title: The "Shadow Architect" (Building the Integrated Agent Environment)

The Context (The Pain): We have entered the "Uncanny Valley" of AI coding. Tools like GitHub Copilot, Cursor, and Windsurf have made writing code faster, but they haven't made *building software* easier. They suffer from three critical failures that prevent them from being true "Co-Founders":

1. **Context Amnesia (The Goldfish Memory):** Current AI tools are "stateless." They forget your architectural decisions, library choices, and project nuances the moment you close the chat session. You find yourself repeating, *"Use Tailwind, not CSS Modules"* or *"Remember we are using Supabase"* every single day. They don't have a "brain"; they just have a buffer.
2. **Folder Pollution (The Hygiene Crisis):** To fix this amnesia, current tools force you to clutter your clean codebase with garbage metadata. You have to commit files like .cursorrules, .gemini, .vscode/settings, or docs/context.md just to give the AI a memory. This "Folder Pollution" is a nightmare for clean version control and team collaboration.
3. **The Security Leak (The .env Nightmare):** This is the biggest blocker for enterprise adoption. Most AI agents recklessly read your entire project directory including your .env files. They treat your AWS Secret Keys and Database Passwords as plain text, potentially sending them to cloud servers (OpenAI/Anthropic) for processing. We are one hallucination away from a major data breach.
4. **The Integration Gap (Code vs. Build):** Current tools are just "Text Editors with a Chatbot." They can *write* the code for a database connection, but they cannot *spin up* the Docker container, *run* the migration, or *debug* the connection error when it fails. They are passive, not active.

The Challenge: Stop building Chatbots. Start building an Environment.

We challenge you to move beyond simple "Autocomplete" and build a **"Local-First Integrated Agent Environment (IAE)"**, a VS Code Extension or a Standalone Editor (Fork) that solves the following engineering hurdles:

- **Solve the "Zero-Footprint" Memory:** How can your AI remember the entire history and architecture of a project (even if I delete and re-clone the folder) *without* creating a single config file inside the project directory? We need a "Shadow Workspace" that persists context globally and invisibly.
- **Solve the "Zero-Trust" Security:** How can your Agent write code that uses API keys and Database secrets *without* ever actually seeing the raw values? Build a system where

the AI acts as a blind orchestrator, injecting secrets at runtime safely without leaking them to the LLM context window.

- **Solve the "Autonomous Integration":** We don't just want code generation; we want **Execution**. Your Agent must be capable of "Mission Control" autonomously running terminal commands to set up environments, connecting local databases, running unit tests, and iteratively fixing its own bugs until the build passes (TDD Loop).
- **Model Agnosticism:** The system must not be locked to one provider. It should allow a "Bring Your Own Key" (BYOK) approach, letting users plug in GPT-4, Claude 3.5, or even a local Ollama model to drive the logic.

The One-Line Deliverable:

> **Build a secure, context-aware AI Agent that acts as a Project Manager, capable of planning, coding, testing, and wiring integrations autonomously while keeping the user's codebase 100% clean of metadata and secrets.**

---

2) Problem Statement Title: Project Aegis – The "Sovereign" Agentic Browser

The Context (The Pain): The modern web browser has betrayed its original purpose.

1. **The "Passive Viewport" Trap:** Browsers like Chrome have become nothing more than "Passive TVs" for advertisers. They prioritize rendering tracking scripts over user intent. They are designed to *show* you content, not *work* for you.
2. **The Agentic Privacy Nightmare:** Emerging "AI Browsers" (like MultiOn or Arc Max) promise automation, but at a terrible cost. To function, they stream your **Session Cookies**, **Auth Tokens**, and **Private PII** to cloud servers (OpenAI/Anthropic). They effectively break the privacy model of the web by centralizing your browsing data.
3. **The "Vibe Coding" Gap:** Non-coders want to automate their digital lives (e.g., *"Login to the portal, download the last 3 invoices, and save them to Drive"*), but they cannot write complex Selenium or Playwright scripts to do it.

The Challenge: Reclaim the User Agent. Build a "Doing" Browser, not a "Viewing" Browser.

We challenge you to build a **Local-First Agentic Browser** (using Tauri v2, Electron, or a Chromium Fork) that transforms the browser from a passive window into an **Autonomous Agent**. It must empower users to execute complex web workflows using natural language **without a single byte of data leaving their device.**

Core Requirements (The "Must-Haves"):

1. The "Sovereign Brain" (Local Vision & Logic)

- **The Problem:** Cloud agents are leakage vectors. Your banking session should never touch an OpenAI server.
- **The Constraint:** Integrate a **Local LLM** (e.g., Llama-3.2-Vision, Phi-3, or Mistral via Ollama) directly into the browser core. The Agent must be able to "see" the current tab (via Screenshots or DOM parsing) and make navigation decisions entirely on-device.

2. The "AI-First Sanitizer" (Native Ad-Blocking)

- **The Problem:** The modern web is "noisy." Ads, pop-ups, and trackers confuse visual AI agents, causing hallucinations and wasted processing power.
- **The Constraint:** Implement a **Native Rust-based Ad-Blocker** (using engines like adblock-rust) that sanitizes the web page at the network layer *before* the AI sees it. This isn't just for user comfort; it is a technical necessity to ensure the Agent perceives a "clean" reality.

3. The "Silent Operator" (DOM Agency)

- **The Problem:** Chatbots just talk. We need action.
- **The Constraint:** Implement a secure "Driver" (adapting libraries like browser-use or Playwright) that connects the Local Brain to the Browser DOM.
- **The Workflow:** The user commands: *"Find the cheapest flight to Mumbai on Skyscanner for next Friday."* -> The Browser **autonomously** navigates, inputs dates, handles the calendar picker, parses results, and presents a clean summary table, all while the user watches.

4. The "Sentinel" (Zero-Trust Guardrails)

- **The Problem:** What if the AI hallucinates and clicks "Delete Account" or "Transfer Funds"?
- **The Constraint:** Implement a **Human-in-the-Loop (HITL)** Protocol for high-stakes actions. The browser must classify DOM elements (e.g., "Submit Payment," "Delete") and force **PAUSE** execution to ask for explicit user confirmation before clicking them.

The One-Line Deliverable:

Build a privacy-first browser that uses Local AI to autonomously navigate, scrape, and interact with the web on the user's behalf, ensuring that no session data or cookies ever leave the local machine.

Problem Statement Title: Project Veracity, The "Self-Healing" Hallucination Firewall

The Context (The Pain): Large Language Models (LLMs) are confident liars. Enterprises want to deploy AI for customer support, financial analysis, and legal drafting, but they cannot trust a model that invents numbers or policies out of thin air.

- **The Enterprise Trust Deficit:** A single hallucinated financial metric or fake legal precedent can cost a company millions in lawsuits and reputation damage. Until this is fixed, Enterprise AI is stuck in the "toy" phase.
- **The Latency Trap:** Current fact-checking solutions are slow. They wait for the LLM to generate the entire response, run a secondary verification prompt, and then return the result. This ruins the real-time, streaming user experience.
- **The "Useless Flag" Problem:** Simply flagging a response as "Potentially Incorrect" leaves the user frustrated. They don't want a warning; they want the correct answer.

The Challenge: Stop building prompt wrappers. Build a Real-Time Defense Layer. We challenge you to build a "Self-Healing Reverse-RAG" (Retrieval-Augmented Generation) firewall. Your system must sit between the LLM and the user, intercepting the streaming output token-by-token, fact-checking claims against a verified local database, and actively mutating/correcting the text before it renders on the user's screen.

Core Requirements (The "Must-Haves"):

1. The Streaming Interceptor (Token-by-Token Analysis)

- **The Problem:** Waiting for the full generation to finish destroys the UX.
- **The Constraint:** Your firewall must process the LLM's Server-Sent Events (SSE) stream in real-time. It needs to detect "claim boundaries" (e.g., when the AI states a specific number, name, or policy) on the fly without adding massive latency.

2. The Reverse-RAG Verification Engine

- **The Problem:** How do you know what the truth is instantly?
- **The Constraint:** Instead of just feeding context into the prompt, build a local Vector Database (the "Ground Truth" vault). As the LLM streams a claim, the firewall must instantly perform a semantic search against the Vault to verify the factual accuracy of that specific sentence.

3. The "Self-Healing" Auto-Correct UI

- **The Problem:** Blocking the output is bad UX.
- **The Constraint:** If a hallucination is detected, the firewall must intercept the lie, retrieve the actual fact from the Vector DB, and seamlessly rewrite the sentence in the stream. The frontend UI must visually highlight this intervention (e.g., highlighting the text in green with a tooltip saying [Auto-Corrected via Verified Source: Document_Name.pdf]).

The One-Line Deliverable

Build a low-latency, real-time AI firewall that intercepts streaming LLM outputs, fact-checks claims against a local ground-truth database, and auto-corrects hallucinations before they reach the user's screen.